

Как да напишем studentbook.cpp

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · list, успоредни списъци, итератори

Какво ще направим

Студентска книжка: един студент, списък от дисциплини и списък с точки по тях. Програмата превръща точките в оценки, изчислява среден успех, намира най-високия и най-ниския резултат.

Новото спрямо упражнения 9 и 10 е контейнерът `list` — и това, че данните се пазят в **два успоредни списъка**.

	vector	list
Устройство	един непрекъснат блок памет	верига от възли, всеки с указател към следващия
<code>v[i]</code>	има	няма
Достъп до <i>i</i> -тия	веднага	чрез обхождане отначало
Вмъкване в средата	измества всичко след мястото	само пренарежда два указателя
Сортиране	<code>std::sort</code>	<code>lst.sort()</code> — собствена функция

План на работата

1. Нов проект, скелет и входният файл
2. Класът `CStudent` (изисквания I.1–I.6)
3. Класът `CStudentBook` и двата успоредни списъка
4. Скалата точки → оценка: `calcScoresList`
5. Извеждане с два итератора наведнъж (изискване II.3)
6. Четене от поток и конструктор по файл (изисквания II.2, II.9)
7. Макс и мин точки (изисквания II.4–II.5)
8. `addPoints` и `calcAverage` (изисквания II.6–II.7)
9. Главна функция (изискване III)

1 Нов проект, скелет и входният файл

Нов **Empty Project**, например `Tema11_Book`, файл `studentbook.cpp`.

```
#include <algorithm> // max_element, min_element, find
#include <cstdio>     // getchar
#include <fstream>
#include <iomanip>
```

```
#include <iostream>
#include <list>
#include <numeric>    // accumulate
#include <stdexcept>  // runtime_error
#include <string>
using namespace std;
```

В папката на проекта — `Тема11.txt` с данните от условието:

```
61462103 0.0
OOP1 20
OOP2 40
OOP3 60
OOP4 80
OOP5 100
```

Пренесете и двете помощни функции за кирилица от упражнение 6, плюс една нова за подравняване вдясно:

```
// Подравняване ВДЯСНО – за заглавията на числовите колони.
static string padLeft(const string& str, size_t nWidth)
{
    size_t nLen = utf8Length(str);
    if (nLen >= nWidth) return str;
    return string(nWidth - nLen, ' ') + str;
}
```

2 Класът CStudent

```
class CStudent
{
private:
    string m_strFN;    // факултетен номер
    double m_dGrade;   // успех

public:
    CStudent();                // I.1
    CStudent(const string& strFN, double dGrade);    // I.2

    const string& getFN() const;
    double getGrade() const;

    void setGrade(double dGrade);                // I.3 презапис на успех
    CStudent& operator=(const CStudent& stud);    // I.6

    friend ostream& operator<<(ostream& toStream, const CStudent& stud); // I.4
    friend istream& operator>>(istream& fromStream, CStudent& stud);    // I.5
};
```

ПРЕЗАПИСЪТ НА УСПЕХ

```
void CStudent::setGrade(double dGrade)
{
    if (dGrade < 2.0 || dGrade > 6.0)
    {
```

```

        if (dGrade != 0.0)          // 0.0 значи "още няма успех" – допустимо е
        {
            cout << " (отхвърлен успех: " << dGrade << ")" << endl;
            return;
        }
    }
    m_dGrade = dGrade;
}

```

Защо 0.0 е изключение от проверката. Файлът от условието започва с 61462103 0.0 — нула значи „успехът още не е изчислен“. Ако проверката отхвърляше и нея, конструкторът по файл нямаше да работи. Такива „специални“ стойности винаги се документират с коментар, иначе след месец никой не помни защо ги има.

ОПЕРАТОРЪТ ЗА ПРИСВОЯВАНЕ

```

// Класът няма указатели, така че генерираният би бил коректен.
// Пишем го, защото условието го изисква.
CStudent& CStudent::operator=(const CStudent& stud)
{
    if (this == &stud) return *this;
    m_strFN = stud.m_strFN;
    m_dGrade = stud.m_dGrade;
    return *this;
}

```

Правилата са от упражнение 3: проверка за самоприсвояване, връщане на `*this` по референция. Тук освобождаване на памет няма — няма какво да се освобождава.

3 Класът CStudentBook и двата успоредни списъка

```

class CStudentBook
{
private:
    CStudent    m_student;          // обект от клас I
    list<string> m_lstSubjects;      // имена на дисциплини
    list<int>    m_lstPoints;        // точки, СЪОТВЕТСТВАЩИ на имената

    list<int> calcScoresList() const; // частна – вътрешна подробност

public:
    CStudentBook();
    explicit CStudentBook(const CStudent& stud);          // II.1
    explicit CStudentBook(const string& strFileName);      // II.2
    ...
};

```

Два успоредни списъка са чуплива конструкция. Условието изисква точно това, затова така го пишем — но си струва да се знае защо в реален код се избягва.

Връзката между „ООРЗ“ и „60“ съществува само дотолкова, доколкото двата елемента са на еднаква позиция. Нищо в кода не я налага. Достатъчно е една функция да добави в единия списък и да пропусне другия, и всички дисциплини след това място получават чужди точки. Компиляторът не може да помогне — и двата списъка са напълно валидни поотделно.

Затова **всяка** функция, която пипа списъците, ги пипа заедно. В `addPoints` (стъпка 8) двата `push_back` стоят един до друг нарочно.

Естественото решение би било `list<pair<string, int>>` или `map<string, int>` — тогава двойката е неразделима по построение. Асоциативните контейнери са темата на упражнения 12 и 13.

explicit пред конструкторите с един параметър. Без него редът `CStudentBook book = stud;` би се компилирал — компилаторът тихомълком би преобразувал студента в книжка. С `explicit` се изисква изричното `CStudentBook book(stud);`. Правилото: конструктор с един параметър се пише `explicit`, освен ако преобразуването наистина не се желае.

4 Скалата точки → оценка

```
// Скалата е обявена на едно място, за да не се повтаря из кода.
list<int> CStudentBook::calcScoresList() const
{
    list<int> lstScores;
    for (list<int>::const_iterator it = m_lstPoints.begin();
         it != m_lstPoints.end(); ++it)
    {
        int nPts = *it;
        int nScore;
        if (nPts >= 88) nScore = 6;           // отличен
        else if (nPts >= 75) nScore = 5;      // мн. добър
        else if (nPts >= 63) nScore = 4;      // добър
        else if (nPts >= 50) nScore = 3;      // среден
        else nScore = 2;                      // слаб
        lstScores.push_back(nScore);
    }
    return lstScores;
}
```

Точки	Оценка	Данните от условието
88–100	6	ООР5 (100)
75–87	5	ООР4 (80)
63–74	4	—
50–62	3	ООР3 (60)
0–49	2	ООР1 (20), ООР2 (40)

Среден успех: $(2 + 2 + 3 + 5 + 6) / 5 = 3.60$. Проверявайте по това число.

Веригата `else if` е подредена от най-високото надолу. Така всяко условие проверява само долната граница — горната е гарантирана от това, че предишното е било лъжа. Ако подредбата се обърне, всяко условие ще трябва и от двете страни: `nPts >= 50 && nPts <= 62`. Работи, но е двойно повече място за грешка.

Функцията е частна. Отвън се вижда резултатът — оценките в `Output` и средният успех. Как точно се превръщат точките в оценки е вътрешна работа и може да се смени, без да засегне никого.

5 Извеждане с два итератора наведнъж

```
ostream& CStudentBook::Output(ostream& toStream) const
{
    toStream << "Студентска книжка: " << m_student << endl;
    // padLeft, а не setw: заглавията са на кирилица (виж utf8Length)
    toStream << " " << padRight("дисциплина", 14) << padLeft("точки", 7)
        << padLeft("оценка", 8) << endl;

    list<int> lstScores = calcScoresList();

    // ДВА успоредни списъка се обхождат с ДВА итератора едновременно.
    list<string>::const_iterator itSub = m_lstSubjects.begin();
    list<int>::const_iterator itPts = m_lstPoints.begin();
    list<int>::const_iterator itScr = lstScores.begin();

    while (itSub != m_lstSubjects.end() && itPts != m_lstPoints.end())
    {
        toStream << " " << padRight(*itSub, 14)
            << setw(7) << right << *itPts
            << setw(8) << right << *itScr << endl;
        ++itSub;
        ++itPts;
        ++itScr;
    }
    return toStream;
}
```

Ето къде липсата на `operator[]` се усеща. При `vector` това щеше да е обикновен цикъл по индекс: `v1[i]` и `v2[i]`. При `list` индекс няма — трябва толкова итератора, колкото списъка, и всички се придвижват заедно.

Затова трите `++` стоят в края, един под друг: така веднага се вижда, ако някой липсва. Пропуснат `++` значи безкраен цикъл.

Условието на цикъла проверява два края. При коректни данни двата списъка са с еднаква дължина и една проверка би стигнала. Но ако някога се разминат, проверката и на двата предпазва от разадресиране на `end()`.

Забележете и `++itSub`, а не `itSub++`. За итератори на списък разликата е реална: постфиксният прави копие на итератора, преди да го придвижи. Навикът да се пише префиксно не струва нищо.

setw работи за ***itPts** и ***itScr**, защото те са числа — цифрите са по един байт. За ***itSub** и за заглавията се ползва **padRight / padLeft**, защото там може да има кирилица. Смесването изглежда непоследователно, но е точно: всяко на мястото си.

6 Четене от поток и конструктор по файл

```
// Формат: <ф.номер> <успех> CR, после по <дисциплина> <точки> на ред.
istream& operator>>(istream& fromStream, CStudentBook& book)
{
    if (!(fromStream >> book.m_student))
        return fromStream;

    book.m_lstSubjects.clear();
    book.m_lstPoints.clear();

    string strSubject;
    int    nPoints = 0;
    while (fromStream >> strSubject >> nPoints)
    {
        book.m_lstSubjects.push_back(strSubject);
        book.m_lstPoints.push_back(nPoints);
    }
    return fromStream;
}
```

Първият ред се чете с оператора на CStudent — `fromStream >> book.m_student` взема номера и успеха наведнъж. Класовете се четат така, както са построени: всеки си знае своя формат.

Двата `push_back` стоят един до друг — това е правилото от стъпка 3 на практика.

КОНСТРУКТОРЪТ ПО ФАЙЛ

```
CStudentBook::CStudentBook(const string& strFileName)
: m_student(), m_lstSubjects(), m_lstPoints()
{
    ifstream iFile(strFileName.c_str());
    if (!iFile)
        throw runtime_error("Не мога да отворя файла: " + strFileName);

    iFile >> *this;

    if (m_lstSubjects.empty())
        throw runtime_error("Файлът " + strFileName + " не съдържа дисциплини");
}
```

Същата постановка като в упражнение 10: обектът или е валиден, или изобщо не се създава.

7 Макс и мин точки

```
int CStudentBook::getMaxPoints() const
{
    if (m_lstPoints.empty()) return 0;
    return *max_element(m_lstPoints.begin(), m_lstPoints.end());
}

int CStudentBook::getMinPoints() const
{
    if (m_lstPoints.empty()) return 0;
    return *min_element(m_lstPoints.begin(), m_lstPoints.end());
}
```

Тези алгоритми работят и върху list — за разлика от `sort`. Разликата е в това какво искат от итераторите: `max_element` само обхожда напред, а `sort` иска да скача на произволно място. Списъкът предлага първото, но не и второто.

Затова `find`, `count`, `accumulate`, `max_element` и `min_element` работят за всеки контейнер, а `sort` — само за `vector` и `deque`.

Проверката `empty()` е задължителна. Върху празен диапазон `max_element` връща `end()`, а разадресирането на `end()` е грешка — програмата ще прочете чужда памет или ще се срине. Книжка без нито една дисциплина е напълно възможна: точно такава е след конструктора от II.1.

8 addPoints и calcAverage

```
// Ако дисциплината вече я има – точките се ДОБАВЯТ към нейните.
// Ако я няма – добавя се нов ред в двата списъка.
void CStudentBook::addPoints(const string& strSubject, int nPoints)
{
    list<string>::iterator itSub = m_lstSubjects.begin();
    list<int>::iterator itPts = m_lstPoints.begin();

    while (itSub != m_lstSubjects.end() && itPts != m_lstPoints.end())
    {
        if (*itSub == strSubject)
        {
            *itPts += nPoints;           // намерена – добавяме
            return;
        }
        ++itSub;
        ++itPts;
    }

    // Не е намерена – добавяме нова двойка В ДВАТА списъка.
    m_lstSubjects.push_back(strSubject);
    m_lstPoints.push_back(nPoints);
}
```

Защо не се ползва `find`. `find(m_lstSubjects.begin(), ..., strSubject)` би намерил дисциплината, но би върнал итератор към **списъка с имена**. От него няма как да се стигне до съответния елемент в списъка с точки — при `vector` щеше да се извади индекс, при `list` това не работи.

Затова обхождаме ръчно с два итератора: когато `itSub` сочи намерената дисциплина, `itPts` сочи нейните точки. Пак същият модел като в стъпка 5.

`itPts` тук е обикновен `iterator`, не `const_iterator`. Редът `*itPts += nPoints;` променя елемента. През `const_iterator` това е забранено и компилаторът дава C3892. Правилото: `const_iterator` за четене, `iterator` за промяна.

СРЕДНИЯТ УСПЕХ

```
// Изчислява средното от ОЦЕНКИТЕ и го записва в член-променливата студент.
double CStudentBook::calcAverage()
{
    list<int> lstScores = calcScoresList();
    if (lstScores.empty())
    {
        m_student.setGrade(0.0);
        return 0.0;
    }

    int nSum = accumulate(lstScores.begin(), lstScores.end(), 0);
    double dAvg = static_cast<double>(nSum) / lstScores.size();

    m_student.setGrade(dAvg);           // записва се в обекта студент
    return dAvg;
}
```

`static_cast<double>(nSum)` не е излишно. `nSum` е `int`, `lstScores.size()` — цяло число. Без преобразуването `18 / 5` се смята като **целочислено деление** и дава `3`, а не `3.6`. Достатъчно е единият операнд да е `double`, за да мине делението по правилата за дробни числа.

Това е една от най-честите грешки при изчисляване на средни стойности и не се забелязва лесно — резултатът е правдоподобен, само малко нисък.

Средното е от оценките, не от точките. Условието казва „средния успех на студента по **оценките** на всички дисциплини“. Средното от точките $(20+40+60+80+100)/5 = 60$ би било съвсем друго число и не би било успех. Затова първият ред вика `calcScoresList()`.

9 Главна функция

9.1–9.2 Създаване и добавяне на точки

```
cout << fixed << setprecision(2);

CStudent stud("61462103", 0.0);
CStudentBook book(stud);           // II.1 – по обект студент
```

```
book.addPoints("00P1", 20);
book.addPoints("00P2", 40);
book.addPoints("00P3", 60);
book.addPoints("00P4", 80);
book.addPoints("00P5", 100);
```

9.3 Проверка, че `addPoints` наистина добавя

```
book.addPoints("00P1", 45);
cout << " 00P1 вече има " << book.getPoints().front() << " точки" << endl;
cout << " брой дисциплини остава: " << book.getSubjects().size() << endl;

// Връщаме стойността, за да съвпадат следващите тестове с условието
book.addPoints("00P1", -45);
```

20 + 45 = 65, а броят дисциплини остава 5 — значи не е добавен нов ред, а е променен съществуващият. После връщаме 20, за да съвпадат по-нататъшните числа с данните от условието.

9.4 Средният успех се записва в студента

```
cout << " успех преди: " << book.getStudent().getGrade() << endl;
double dAvg = book.calcAverage();
cout << " calcAverage() върна: " << dAvg << endl;
cout << " успех след: " << book.getStudent().getGrade()
    << " ← записан в обекта студент" << endl;
```

Трите реда доказват изискване II.7: функцията не само връща числото, а го и **записва**. Преди извикването успехът е 0.00, след него — 3.60.

9.5 Създаване по файл и извеждане във файл

```
CStudentBook bookFile("Tema11.txt");
cout << bookFile;

bookFile.calcAverage();

ofstream oFile("Tema11_out.txt");
if (oFile)
{
    oFile << fixed << setprecision(2);
    oFile << bookFile;                // СЪЩИЯТ оператор
    oFile.close();
}
```

Същото, което видяхме в упражнение 4: операторът приема `ostream&`, а файловият поток е негов наследник. Нито ред нов код за записване във файл.

Форматирането се задава отделно за всеки поток — `setprecision` на `cout` не важи за `oFile`.

9.6 Собствените функции на `list`

```
list<int> lstPts = book.getPoints();

// list няма operator[] – обхожда се само с итератори
for (list<int>::const_iterator it = lstPts.begin(); it != lstPts.end(); ++it)
    cout << *it << " ";

// list има СОБСТВЕН sort – std::sort не работи върху него
lstPts.sort();
lstPts.reverse();
```

`sort(lstPts.begin(), lstPts.end())` не се компилира. Грешката е дълга и неясна — страница шаблонни съобщения за `random_access_iterator_tag`. Причината е проста: `std::sort` иска да скача на произволен елемент, а списъкът може само стъпка по стъпка.

Затова `list` има собствени `sort()`, `reverse()`, `unique()`, `merge()` и `remove()` — те работят чрез пренареждане на указатели и не местят данни. Викат се като функции на обекта: `lstPts.sort()`;

Компилиране и стартиране

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build.
<code>F5</code>	Start Debugging. Работната папка е тази на проекта — оттам се търси <code>Tema11.txt</code> .
<code>F9</code>	Точка на прекъсване в цикъла на <code>Output</code> . В Watch следете <code>*itSub</code> и <code>*itPts</code> — вижда се как двата итератора вървят в крак.

```
cl /EHsc /std:c++17 /utf-8 studentbook.cpp
studentbook.exe
```

Кирилица под Windows

Файлът се записва като *Unicode (UTF-8 with signature)* — *Codepage 65001*; компилаторът получава `/utf-8`; конзолата се превключва с `chcp 65001` или със `system("chcp 65001 > nul");` като първи ред в `main`.

Tema11_out.txt се записва в **UTF-8**, защото програмата извежда кирилски заглавия. Notepad го отваря правилно.

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 11: STL, контейнери, итератори, алгоритми.
// CStudent      – факултетен номер и успех.
// CStudentBook  – студентска книжка: студент + два УСПОРЕДНИ списъка
//                (дисциплини и точки по тях).

#include <algorithm> // max_element, min_element, find
#include <cstdio>    // getchar
#include <fstream>
#include <iomanip>
#include <iostream>
#include <list>
#include <numeric>   // accumulate
#include <stdexcept> // runtime_error
#include <string>
using namespace std;

// =====
// Помощно: подравняване при кирилица (setw брои байтове)
// =====
static size_t utf8Length(const string& str)
{
    size_t nLen = 0;
    for (size_t i = 0; i < str.length(); i++)
        if ((static_cast<unsigned char>(str[i]) & 0xC0) != 0x80)
            nLen++;
    return nLen;
}

static string padRight(const string& str, size_t nWidth)
{
    size_t nLen = utf8Length(str);
    if (nLen >= nWidth) return str;
    return str + string(nWidth - nLen, ' ');
}

// Подравняване ВДЯСНО – за заглавията на числовите колонии.
static string padLeft(const string& str, size_t nWidth)
{
    size_t nLen = utf8Length(str);
    if (nLen >= nWidth) return str;
    return string(nWidth - nLen, ' ') + str;
}

// =====
// I. Клас CStudent
// =====
class CStudent
{
private:
    string m_strFN;      // факултетен номер
    double m_dGrade;     // успех
```

```

public:
    CStudent(); // I.1
    CStudent(const string& strFN, double dGrade); // I.2

    const string& getFN() const;
    double getGrade() const;

    void setGrade(double dGrade); // I.3 презапис на успех

    CStudent& operator=(const CStudent& stud); // I.6

    friend ostream& operator<<(ostream& toStream, const CStudent& stud); // I.4
    friend istream& operator>>(istream& fromStream, CStudent& stud); // I.5
};

CStudent::CStudent() : m_strFN("(няма)"), m_dGrade(0.0) {}

CStudent::CStudent(const string& strFN, double dGrade)
    : m_strFN(strFN), m_dGrade(dGrade) {}

const string& CStudent::getFN() const { return m_strFN; }
double CStudent::getGrade() const { return m_dGrade; }

// ----- I.3 Презапис на успех -----
void CStudent::setGrade(double dGrade)
{
    if (dGrade < 2.0 || dGrade > 6.0)
    {
        if (dGrade != 0.0) // 0.0 значи "още няма успех" – допустимо е
        {
            cout << " (отхвърлен успех: " << dGrade << ")" << endl;
            return;
        }
    }
    m_dGrade = dGrade;
}

// ----- I.6 Оператор за присвояване -----
// Класът няма указатели, така че генерираният би бил коректен.
// Пишем го, защото условието го изисква.
CStudent& CStudent::operator=(const CStudent& stud)
{
    if (this == &stud) return *this;
    m_strFN = stud.m_strFN;
    m_dGrade = stud.m_dGrade;
    return *this;
}

// ----- I.4 Извеждане -----
ostream& operator<<(ostream& toStream, const CStudent& stud)
{
    toStream << "ф.н. " << stud.m_strFN << ", успех " << fixed << setprecision(2)
        << stud.m_dGrade;
    return toStream;
}

// ----- I.5 Четене -----
istream& operator>>(istream& fromStream, CStudent& stud)
{

```

```

    string strFN;
    double dGrade = 0.0;
    if (fromStream >> strFN >> dGrade)
    {
        stud.m_strFN = strFN;
        stud.m_dGrade = dGrade;
    }
    return fromStream;
}

// =====
// II. Клас CStudentBook
// =====
class CStudentBook
{
private:
    CStudent      m_student;          // обект от клас I
    list<string> m_lstSubjects;        // имена на дисциплини
    list<int>     m_lstPoints;         // точки, СЪОТВЕТСТВАЩИ на имената

    // Изчислява и връща списък с оценки по дисциплините
    list<int> calcScoresList() const;

public:
    CStudentBook();
    explicit CStudentBook(const CStudent& stud);          // II.1
    explicit CStudentBook(const string& strFileName);     // II.2

    const CStudent& getStudent() const;
    const list<string>& getSubjects() const;
    const list<int>& getPoints() const;

    ostream& Output(ostream& toStream) const;             // II.3

    int getMaxPoints() const;                             // II.4
    int getMinPoints() const;                             // II.5

    void addPoints(const string& strSubject, int nPoints); // II.6
    double calcAverage();                                 // II.7

    friend ostream& operator<<(ostream& toStream, const CStudentBook& book); // II.8
    friend istream& operator>>(istream& fromStream, CStudentBook& book);    // II.9
};

CStudentBook::CStudentBook()
    : m_student(), m_lstSubjects(), m_lstPoints() {}

// ----- II.1 Експлицитен конструктор с обект студент -----
CStudentBook::CStudentBook(const CStudent& stud)
    : m_student(stud), m_lstSubjects(), m_lstPoints() {}

// ----- II.2 Експлицитен конструктор с име на файл -----
CStudentBook::CStudentBook(const string& strFileName)
    : m_student(), m_lstSubjects(), m_lstPoints()
{
    ifstream iFile(strFileName.c_str());
    if (!iFile)
        throw runtime_error("Не мога да отворя файла: " + strFileName);
}

```

```

iFile >> *this;

if (m_lstSubjects.empty())
    throw runtime_error("Файлът " + strFileName + " не съдържа дисциплини");
}

const CStudent& CStudentBook::getStudent() const { return m_student; }
const list<string>& CStudentBook::getSubjects() const { return m_lstSubjects; }
const list<int>& CStudentBook::getPoints() const { return m_lstPoints; }

// ----- Точки -> оценка -----
// Скалата е обявена на едно място, за да не се повтаря из кода.
list<int> CStudentBook::calcScoresList() const
{
    list<int> lstScores;
    for (list<int>::const_iterator it = m_lstPoints.begin();
        it != m_lstPoints.end(); ++it)
    {
        int nPts = *it;
        int nScore;
        if (nPts >= 88) nScore = 6; // отличен
        else if (nPts >= 75) nScore = 5; // мн. добър
        else if (nPts >= 63) nScore = 4; // добър
        else if (nPts >= 50) nScore = 3; // среден
        else nScore = 2; // слаб
        lstScores.push_back(nScore);
    }
    return lstScores;
}

// ----- II.3 Извеждане -----
ostream& CStudentBook::Output(ostream& toStream) const
{
    toStream << "Студентска книжка: " << m_student << endl;
    // padLeft, а не setw: заглавията са на кирилица (виж utf8Length)
    toStream << " " << padRight("дисциплина", 14) << padLeft("точки", 7)
        << padLeft("оценка", 8) << endl;

    list<int> lstScores = calcScoresList();

    // ДВА успоредни списъка се обхождат с ДВА итератора едновременно.
    list<string>::const_iterator itSub = m_lstSubjects.begin();
    list<int>::const_iterator itPts = m_lstPoints.begin();
    list<int>::const_iterator itScr = lstScores.begin();

    while (itSub != m_lstSubjects.end() && itPts != m_lstPoints.end())
    {
        toStream << " " << padRight(*itSub, 14)
            << setw(7) << right << *itPts
            << setw(8) << right << *itScr << endl;
        ++itSub;
        ++itPts;
        ++itScr;
    }
    return toStream;
}

ostream& operator<<(ostream& toStream, const CStudentBook& book)
{

```

```

        return book.Output(toStream);
    }

// ----- II.9 Четене -----
// Формат: <ф.номер> <успех> CR, после по <дисциплина> <точки> на ред.
istream& operator>>(istream& fromStream, CStudentBook& book)
{
    if (!(fromStream >> book.m_student))
        return fromStream;

    book.m_lstSubjects.clear();
    book.m_lstPoints.clear();

    string strSubject;
    int    nPoints = 0;
    while (fromStream >> strSubject >> nPoints)
    {
        book.m_lstSubjects.push_back(strSubject);
        book.m_lstPoints.push_back(nPoints);
    }
    return fromStream;
}

// ----- II.4 Максимални точки -----
int CStudentBook::getMaxPoints() const
{
    if (m_lstPoints.empty()) return 0;
    return *max_element(m_lstPoints.begin(), m_lstPoints.end());
}

// ----- II.5 Минимални точки -----
int CStudentBook::getMinPoints() const
{
    if (m_lstPoints.empty()) return 0;
    return *min_element(m_lstPoints.begin(), m_lstPoints.end());
}

// ----- II.6 Добавяне на точки по дисциплина -----
// Ако дисциплината вече я има – точките се ДОБАВЯТ към нейните.
// Ако я няма – добавя се нов ред в двата списъка.
void CStudentBook::addPoints(const string& strSubject, int nPoints)
{
    list<string>::iterator itSub = m_lstSubjects.begin();
    list<int>::iterator    itPts = m_lstPoints.begin();

    while (itSub != m_lstSubjects.end() && itPts != m_lstPoints.end())
    {
        if (*itSub == strSubject)
        {
            *itPts += nPoints;          // намерена – добавяме
            return;
        }
        ++itSub;
        ++itPts;
    }

    // Не е намерена – добавяме нова двойка В ДВАТА списъка.
    m_lstSubjects.push_back(strSubject);
    m_lstPoints.push_back(nPoints);
}

```

```

}

// ----- II.7 Среден успех -----
// Изчислява средното от ОЦЕНКИТЕ и го записва в член-променливата студент.
double CStudentBook::calcAverage()
{
    list<int> lstScores = calcScoresList();
    if (lstScores.empty())
    {
        m_student.setGrade(0.0);
        return 0.0;
    }

    int nSum = accumulate(lstScores.begin(), lstScores.end(), 0);
    double dAvg = static_cast<double>(nSum) / lstScores.size();

    m_student.setGrade(dAvg);          // записва се в обекта студент
    return dAvg;
}

// =====
// III. Главна функция
// =====
int main()
{
    cout << fixed << setprecision(2);

    // ===== III.1 Създаване на обекти от I и II =====
    cout << "===== III.1 Създаване на обекти =====" << endl;
    CStudent stud("61462103", 0.0);
    cout << "    студент: " << stud << endl;

    CStudentBook book(stud);          // II.1 – по обект студент
    cout << "    книжка, създадена по обект студент:" << endl;
    cout << book;

    // ===== III.2 Добавяне на точки по 5 дисциплини =====
    cout << endl << "===== III.2 Добавяне на точки (addPoints) =====" << endl;
    book.addPoints("00P1", 20);
    book.addPoints("00P2", 40);
    book.addPoints("00P3", 60);
    book.addPoints("00P4", 80);
    book.addPoints("00P5", 100);
    cout << book;

    // addPoints върху съществуваща дисциплина ДОБАВЯ точки
    cout << "---- addPoints(\"00P1\", 45) върху съществуваща дисциплина ----" << endl;
    book.addPoints("00P1", 45);
    cout << "    00P1 вече има " << book.getPoints().front() << " точки" << endl;
    cout << "    брой дисциплини остава: " << book.getSubjects().size() << endl;

    // Връщаме стойността, за да съвпадат следващите тестове с условието
    book.addPoints("00P1", -45);

    // ===== III.3 Среден успех =====
    cout << endl << "===== III.3 Среден успех (calcAverage) =====" << endl;
    cout << "    успех преди: " << book.getStudent().getGrade() << endl;
    double dAvg = book.calcAverage();
    cout << "    calcAverage() върна: " << dAvg << endl;
}

```

```

cout << "    успех след: " << book.getStudent().getGrade()
    << "    <- записан в обекта студент" << endl;

// ===== III.4 Извеждане на всички данни =====
cout << endl << "===== III.4 Всички данни =====" << endl;
cout << book;

// ===== III.5 и III.6 Макс и мин точки =====
cout << endl << "===== III.5-III.6 Макс и мин точки =====" << endl;
cout << "    максимални точки: " << book.getMaxPoints() << endl;
cout << "    минимални точки: " << book.getMinPoints() << endl;

// ===== III.7 Създаване чрез файл =====
cout << endl << "===== III.7 Създаване чрез файл =====" << endl;

// а) несъществуващ файл
try
{
    CStudentBook bookBad("няма-такъв-файл.txt");
    cout << bookBad;
}
catch (const exception& ex)
{
    cout << "    хванато изключение: " << ex.what() << endl;
}

// б) истинският файл
try
{
    CStudentBook bookFile("Tema11.txt");
    cout << "---- прочетено от Tema11.txt ----" << endl;
    cout << bookFile;

    bookFile.calcAverage();
    cout << "---- след calcAverage() ----" << endl;
    cout << bookFile;
    cout << "    макс: " << bookFile.getMaxPoints()
        << ", мин: " << bookFile.getMinPoints() << endl;

    // Извеждане във файл – същият оператор
    ofstream oFile("Tema11_out.txt");
    if (oFile)
    {
        oFile << fixed << setprecision(2);
        oFile << bookFile;
        oFile.close();
        cout << "    записано и в Tema11_out.txt" << endl;
    }
}
catch (const exception& ex)
{
    cout << "    хванато изключение: " << ex.what() << endl;
}

// ===== Операции със списъка =====
cout << endl << "===== Операции с list =====" << endl;
list<int> lstPts = book.getPoints();
cout << "    size()    = " << lstPts.size() << endl;
cout << "    front()   = " << lstPts.front() << endl;

```

```

cout << "  back()   = " << lstPts.back() << endl;

// list няма operator[] – обхожда се само с итератори
cout << "  съдържание: ";
for (list<int>::const_iterator it = lstPts.begin(); it != lstPts.end(); ++it)
    cout << *it << " ";
cout << endl;

// list има СОБСТВЕН sort – std::sort не работи върху него
lstPts.sort();
cout << "  след lstPts.sort(): ";
for (list<int>::const_iterator it = lstPts.begin(); it != lstPts.end(); ++it)
    cout << *it << " ";
cout << endl;

lstPts.reverse();
cout << "  след reverse():   ";
for (list<int>::const_iterator it = lstPts.begin(); it != lstPts.end(); ++it)
    cout << *it << " ";
cout << endl;

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

Очакван изход

```

===== III.1 Създаване на обекти =====
студент: ф.н. 61462103, успех 0.00
книжка, създадена по обект студент:
Студентска книжка: ф.н. 61462103, успех 0.00
дисциплина      точки  оценка

===== III.2 Добавяне на точки (addPoints) =====
Студентска книжка: ф.н. 61462103, успех 0.00
дисциплина      точки  оценка
00P1             20     2
00P2             40     2
00P3             60     3
00P4             80     5
00P5            100     6
--- addPoints("00P1", 45) върху съществуваща дисциплина ---
00P1 вече има 65 точки
брой дисциплини остава: 5

===== III.3 Среден успех (calcAverage) =====
успех преди: 0.00
calcAverage() върна: 3.60
успех след:  3.60  <- записан в обекта студент

===== III.4 Всички данни =====
Студентска книжка: ф.н. 61462103, успех 3.60
дисциплина      точки  оценка
00P1             20     2
00P2             40     2

```

```

00P3          60      3
00P4          80      5
00P5         100      6

===== III.5-III.6 Макс и мин точки =====
максимални точки: 100
минимални точки:  20

===== III.7 Създаване чрез файл =====
хванато изключение: Не мога да отворя файла: няма-такъв-файл.txt
--- прочетено от Tema11.txt ---
Студентска книжка: ф.н. 61462103, успех 0.00
дисциплина    точки  оценка
00P1           20     2
00P2           40     2
00P3           60     3
00P4           80     5
00P5          100     6
--- след calcAverage() ---
Студентска книжка: ф.н. 61462103, успех 3.60
дисциплина    точки  оценка
00P1           20     2
00P2           40     2
00P3           60     3
00P4           80     5
00P5          100     6
макс: 100, мин: 20
записано и в Tema11_out.txt

===== Операции с list =====
size()  = 5
front() = 20
back()  = 100
съдържание: 20 40 60 80 100
след lstPts.sort(): 20 40 60 80 100
след reverse():    100 80 60 40 20

```

Чести грешки

Код	Съобщение	Причина и решение
—	Страница шаблонни грешки около <code>_Iter_cat</code> / <code>random_access_iterator_tag</code>	Използван е <code>std::sort</code> върху <code>list</code> . Пише се <code>lst.sort();</code> (стъпка 9.6).
C2676	binary '[': 'std::list' does not define this operator	<code>list</code> няма <code>operator[]</code> . Обхожда се с итератори.
C3892	you cannot assign to a variable that is const	Променя се елемент през <code>const_iterator</code> . В <code>addPoints</code> трябва обикновен <code>iterator</code> .
C2440	cannot convert from 'const_iterator' to 'iterator'	Функцията е <code>const</code> , но се опитва да променя. Махнете <code>const</code> или не променяйте.
C3861	'accumulate': identifier not found	Липсва <code>#include <numeric></code> .

—	Средният успех излиза 3.00 вместо 3.60	Целочислено деление. Липсва <code>static_cast<double></code> (стъпка 8).
—	Средният успех е 60.00	Осреднени са точките вместо оценките. Първо <code>calcScoresList()</code> .
—	Безкраен цикъл при извеждане	Липсва някой от <code>++</code> в края на тялото (стъпка 5).
—	Дисциплините получават чужди точки	Някъде е добавено само в единия списък. Двата <code>push_back</code> се пишат винаги заедно.
—	Срив при празна книжка	Липсва проверката <code>empty()</code> преди <code>max_element</code> (стъпка 7).
—	Заглавията на колоните се сливат	<code>setw</code> брой байтове. За кирилица — <code>padLeft</code> / <code>padRight</code> .
—	<code>Tema11.txt</code> не се намира	Файлът е в папката с <code>.vcxproj</code> , не в <code>Debug</code> .

Обобщение: `vector` или `list`

Изберете <code>vector</code> , когато	Изберете <code>list</code> , когато
трябва достъп по индекс	често се вмъква и маха в средата
ще сортирате с <code>std::sort</code>	итераторите трябва да останат валидни след промени
обхождате често (данните са едно до друго в паметта и това е забележимо по-бързо)	елементите са големи и преместването им е скъпо

На практика `vector` е разумният избор по подразбиране — включително често и когато има вмъквания в средата, защото непрекъснатата памет компенсира. `list` се избира съзнателно, когато има конкретна причина. Тук причината е учебна: да се види контейнер без произволен достъп и какво следва от това.

Какво следва

Упражнение 12	<code>map</code> и <code>multimap</code> — ключ и стойност в едно. Двата успоредни списъка отук се заменят с един <code>map<string, int></code> .
Упражнение 13	<code>set</code> — множество без повторения, и <code>map<string, list<int>></code> .